Paula FAQ — Operating the Portal

Ari
May 24, 2026

Paula FAQ — operating the portal

Answers to questions from the 5/17 spec + week-of agenda. Living doc; update as new questions come in.

---

1. Consultation form — emails + data access

Q: I assume I'll receive an email when someone fills out the consultation form?

Yes. Every submission at `/contact` fires an email to `ADMIN_NOTIFICATION_EMAIL` via Resend (env var on Vercel). Defaults to Paula + Ari. To change: edit the env var in the Vercel dashboard, redeploy. Subject line example: `New consultation: Jane Smith — Private tutoring`.

If the email isn't arriving, check:

1. Vercel env var is set
2. Resend API key in env (`RESEND_API_KEY`) is valid
3. The recipient address isn't blocking the sender domain
4. Check `/admin/notifications` for the in-portal log entry — if it's there but no email arrived, the issue is downstream of the portal.

Q: How do I access all of the data captured in the form?

Three ways, easiest first:

1. **In the portal** — `/admin/consultations` (new this sprint). Lists every request with searchable filter + per-row contact links + CSV export button. Best for day-to-day.
2. **Raw DynamoDB** — table `mathitude-staging-bookings`, filter on `type = "consultation"`. Use the AWS console (us-west-2) or the AWS CLI. Best for one-off audits.
3. **Future agentic query** — phase-4 work. We're holding off until we see which queries you actually run repeatedly. If `/admin/consultations` misses something specific, tell us; that informs the agentic query schema.

---

2. Database access — GUI, SQL, or agentic?

Q: We'll need to be able to grab all data at any time. How will we do that?

The portal already covers ~95% of read paths via admin pages:

What you need	Where to look
All families	`/admin/families` (searchable card grid)
All students	`/admin/students` (sortable table + filter)
All sessions	`/admin/schedule` (weekly) + `/admin/billing` (queue)
All payments	`/admin/payments`
Financial rollups	`/admin/financials`
Consultation submissions	`/admin/consultations` (NEW)
Admin activity log	`/admin/notifications`
Tutor list	`/admin/tutors`

For the 5% the GUI can't do today:

a) Direct DynamoDB access. AWS account `050451400186`, region `us-west-2`, tables `math-itude-staging-*`. Sara/Nikki have read creds. Limits: no SQL, no JOINs. DynamoDB is key-value + GSIs — works great for "all sessions for family X this month" but bad for "show me families whose average session rate is above $100."

b) CSV export. Already on `/admin/consultations`. Will add to other pages as you ask for them. Cheap to add (1 button per page).

c) Athena (future option). AWS Athena can query DynamoDB exports as SQL. Good for monthly ad-hoc reports. Not wired up yet. Setup cost: ~1 day. Cost: a few dollars a month + per-query cents. Flag this when you want it.

d) Agentic query (future). Phase 4. Natural-language questions ("show me overdue families this month") translated to DynamoDB queries by Claude. Defer until we see what queries you ask repeatedly.

Bottom line: use the portal first; tell us when it stops being enough.

---

3. Pricing — per family / per student / per tutor

Currently the session row stores `rate` + `amountCents` directly, with no "resolve at create time" rule. That means:

- **Default rate per student** — set on the student record (already there)
- **Per-session override** — `/admin/sessions/new` lets you enter any amount (e.g. cross-family billing splits, sliding-scale)
- **Per-tutor rate** — track on the tutor record (planned, not shipped). Until that ships, just enter the per-session amount manually.

The Session row carries a `sessionLeadId` (5/17 spec) so you can mark who actually delivered the session even when it differs from the assigned tutor. Rate reconciliation will use that field once the per-tutor rate table ships.

---

4. Database snapshots / offline access

DynamoDB tables have point-in-time recovery (PITR) enabled. The `mathitude-staging-secrets` table also has PITR. To restore: AWS console → DynamoDB → table → Backups → Restore.

For offline analysis, export to S3 (one-off; takes ~minutes for our volume). Run `aws dynamodb export-table-to-point-in-time` and follow the prompts. Files land as JSON-per-line in S3, ingestable by spreadsheets or notebooks.

---

5. SOC2 compliance

Recommended for handling student + payment data. Not blocking, but if parents/schools ask about compliance posture, the answer is currently “not yet.” Path to SOC2 Type 1 (~3 months):

1. Pick a compliance vendor (Vanta, Drata, Secureframe). ~$10-15K/year
2. They auto-collect controls from AWS + Vercel + Clerk + Stripe
3. ~30 days of evidence collection, then auditor review
4. Type 1 = “controls exist.” Type 2 = “controls work over 6+ months.”

Stripe + Clerk + AWS are all SOC2 Type 2 themselves, so most of the heavy lifting on payment + auth is already covered by sub-processor reports. The work is on portal-side controls: access logs, encryption, incident-response runbooks, vendor reviews.

Cheapest first step: enable AWS CloudTrail + retain logs 365 days; turn on GuardDuty for the AWS account. Costs <$50/mo and is required evidence anyway.

---

6. Typical-day walkthrough

From the agenda: “Have Paula go through a typical day of tutoring. Show process of toggling between Excel sessions tab, Excel scheduling tab, Google sheets, Google calendar and Stripe.”

We need this discovery session. The portal’s job is to replace as many of those tab-toggles as possible. Until we watch you work, every UI decision is partially guesswork. Schedule whenever.

---

7. Default card setting — testing

From the agenda. Two things to test:

1. **In the portal** — `/dashboard/billing` as a parent. Add card → it appears non-default. Click “Set as default” → “— pending default” caption. Click “Save Changes” → moss-tinted “Changes saved.”
2. **In Stripe** — same customer in dashboard. Confirm `customer.invoice_settings.default_payment_method` matches. Test card `4242 4242 4242 4242` in test mode.

QA test IDs cover this end-to-end: T-BILL-04 through T-BILL-09. See `QA_TESTS.md` in the repo root.

---

8. Quick reference — environment variables

Env var	Purpose	Where set
`STRIPE_SECRET_KEY`	Live charging	DDB `mathitude-staging-secrets` (preferred) or Vercel env
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Frontend Elements	Vercel env
`STRIPE_WEBHOOK_SECRET`	Webhook signing	DDB secrets table
`RESEND_API_KEY`	Outgoing email	Vercel env
`ADMIN_NOTIFICATION_EMAIL`	Recipient for portal alerts	Vercel env (comma-separated list ok)
`CLERK_SECRET_KEY` + `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	Auth	Vercel env
`DYNAMODB_TABLE_PREFIX`	Table namespace	Vercel env (defaults to `mathitude-staging`)
`AWS_REGION`	AWS region	Vercel env (defaults to `us-west-2`)

Rotation policy: rotate Stripe + Clerk keys quarterly. Resend can be rotated annually unless suspected exposure. AWS IAM keys: use the App Runner instance role for backend; for the portal (Vercel), prefer short-lived role assumption when we add the option.